Operating Systems Assignment 9 — Report

Question 1: Printing the Page Table

Problem: To implement a system call that prints the page table of the calling process, including virtual addresses, physical addresses, and page flags, along with the total number of valid user pages and page-table pages.

Implementation:

- Implemented `vmprint()` to traverse the page directory and page tables.
- Printed the virtual address, physical address, and Present (P), Writable (W), and User (U) flags for valid mappings.
- Ignored kernel-space addresses and counted the valid user pages and page-table pages.
- Created `pttest.c` to display the page table before and after calling `sbrk(4 * 4096)`.

Result: The output showed 3 valid user pages before allocation and 7 afterward, demonstrating that four additional pages were allocated. The page-table information and flags were printed successfully.

Question 2: Handling Page Faults Using Lazy Allocation

Problem: To delay physical memory allocation until a page is accessed for the first time and handle page faults in the kernel.

Implementation:

- Modified `sys_sbrk()` to increase the logical process size without immediately allocating physical memory.
- Updated `deallocuvm()` and `copyuvm()` to skip pages that have not been allocated.
- Modified the trap handler to allocate and zero-initialize a page when a valid page fault occurs, map it into the process address space, and increment a per-process page-fault counter.
- Implemented the `getfaults()` system call to retrieve the number of handled page faults.
- Created `faulttest.c` to allocate 32 pages, write to five distinct pages, and test an access beyond the allocated region.

Result: The program reported exactly 5 handled page faults after accessing five distinct pages. The invalid-access test terminated the child process while the parent process continued running.

Question 3: FIFO and LRU Page Replacement

Problem: To implement and compare FIFO and LRU page replacement algorithms using a given reference string and different numbers of frames.

Implementation:

- Created `pagerepl.c` to simulate FIFO and LRU page replacement.
- Printed the frame contents after each reference and identified each access as a hit or a page fault.
- Tested the reference string `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1` using 3, 4, and 5 frames.
- Calculated the number of page faults and hit ratios for both algorithms.

Result: With 3 frames, FIFO produced 15 page faults, while LRU produced 12 page faults. The results demonstrate the difference between the two replacement algorithms and how the number of frames affects page faults.

Conclusion

The assignment demonstrated page-table traversal, lazy allocation through page-fault handling, and the comparison of FIFO and LRU page replacement algorithms. The test programs provided the expected results for the implemented functionality.